Fpt Software

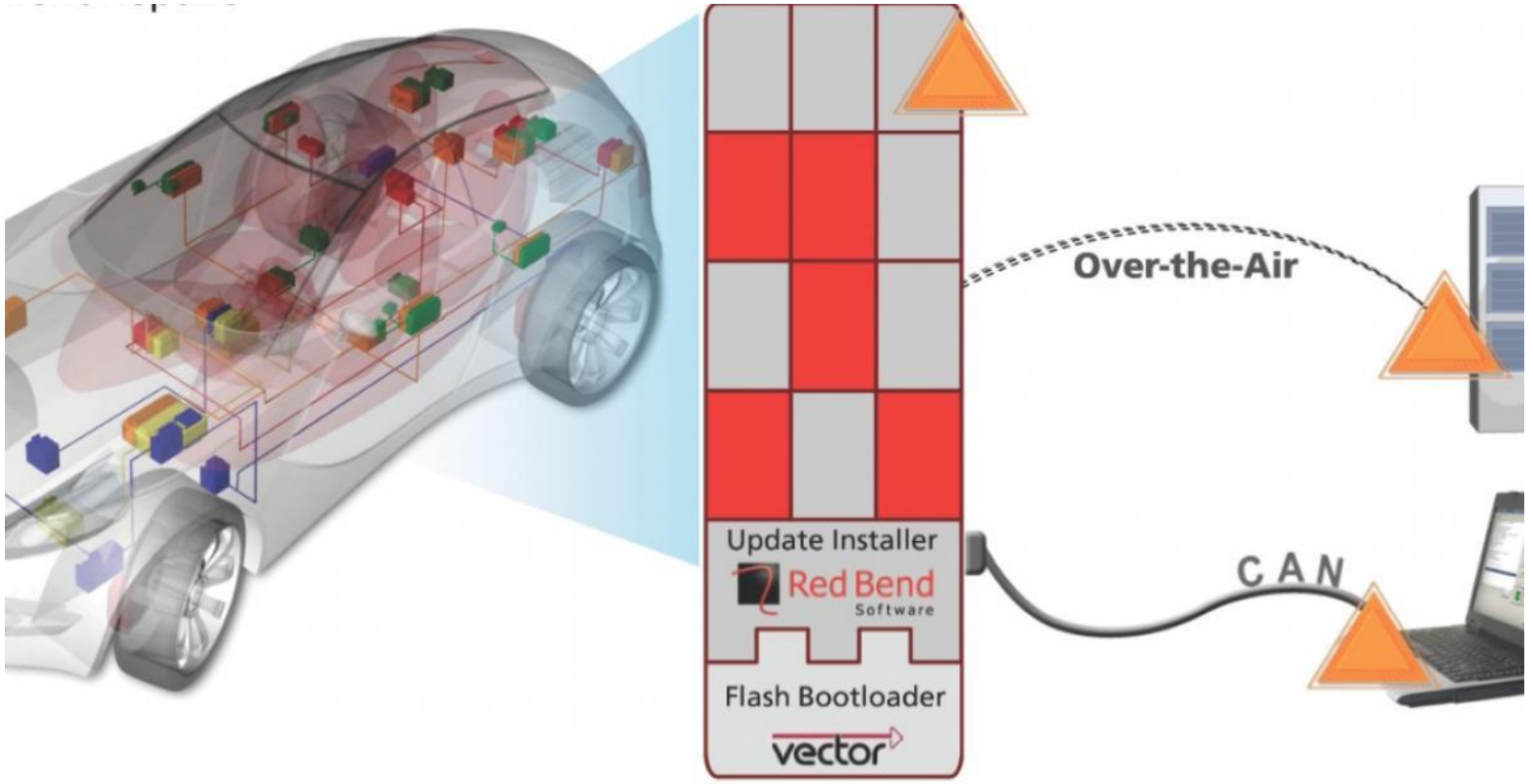
Booting Process & Boot Loader

Our 20TH ANNIVERSARY 1999 - 2019

BOOTING PROCESS & BOOTLOADER

❖**Agenda**

1. Reset source
2. Booting process
3. Boot loader
4. Exercise



1. RESET SOURCE

There are many reset sources in S32K144, this section just describes some typical reset sources.

1. Reset by hardware:
   - Power on reset: reset everything in the microcontroller. This includes the processor and its debug support component and peripherals. In S32K144: When power is initially applied to the MCU or when the supply voltage drops below the POR detect voltage (VPOR), the POR circuit causes a POR reset condition.
   - External pin reset.
   - Low voltage detect (LVD) in S32K144: The LVD system can be configured to generate a reset upon detection of a low voltage condition. After an LVD reset has occurred, the LVD system holds the MCU in reset until the supply voltage has risen above the low voltage detection threshold.
     Besides LVD operation, the device also supports LVR(Low Voltage Reset) operation. If the supply voltage falls below the reset trip point (VLVR), a system reset will be generated.
2. Reset by software:
   - System reset: reset just the processor and peripherals, but not the debug support component of the processor.
   - Processor reset: reset the processor only.
   - Watchdog reset : If the periodic refreshing does not occur, the watchdog issues a system reset.
   - User can generate a system reset or processor reset via a register in the System Control Block (SCB)

Table 7.20 Application Interrupt and Reset Control Register (SCB->AIRCR, Address 0xE000ED0C)

Bits	Name	Type	Reset Value	Description
31:16	VECTKEY	R/W	–	Access key; 0x05FA must be written to this field to write to this register, otherwise the write will be ignored; the read-back value of the upper half-word is 0xFA05
15	ENDIANNESS	R	–	Indicates endianness for data: 1 for big endian (BE8) and 0 for little endian; this can only change after a reset
10:8	PRIGROUP	R/W	0	Priority group
2	SYSRESETREQ	W	–	Requests chip control logic to generate a reset
1	VECTCLRACTIVE	W	–	Clears all active state information for exceptions; typically used in debug or OS to allow system to recover from system error (reset is safer).
0	VECTRESET	W	–	Resets the Cortex®-M3/M4 processor (except debug logic), but this will not reset circuits outside the processor. This is intended for debug operations. Do not use this at the same time as SYSRESETREQ.

2. BOOTING PROCESS

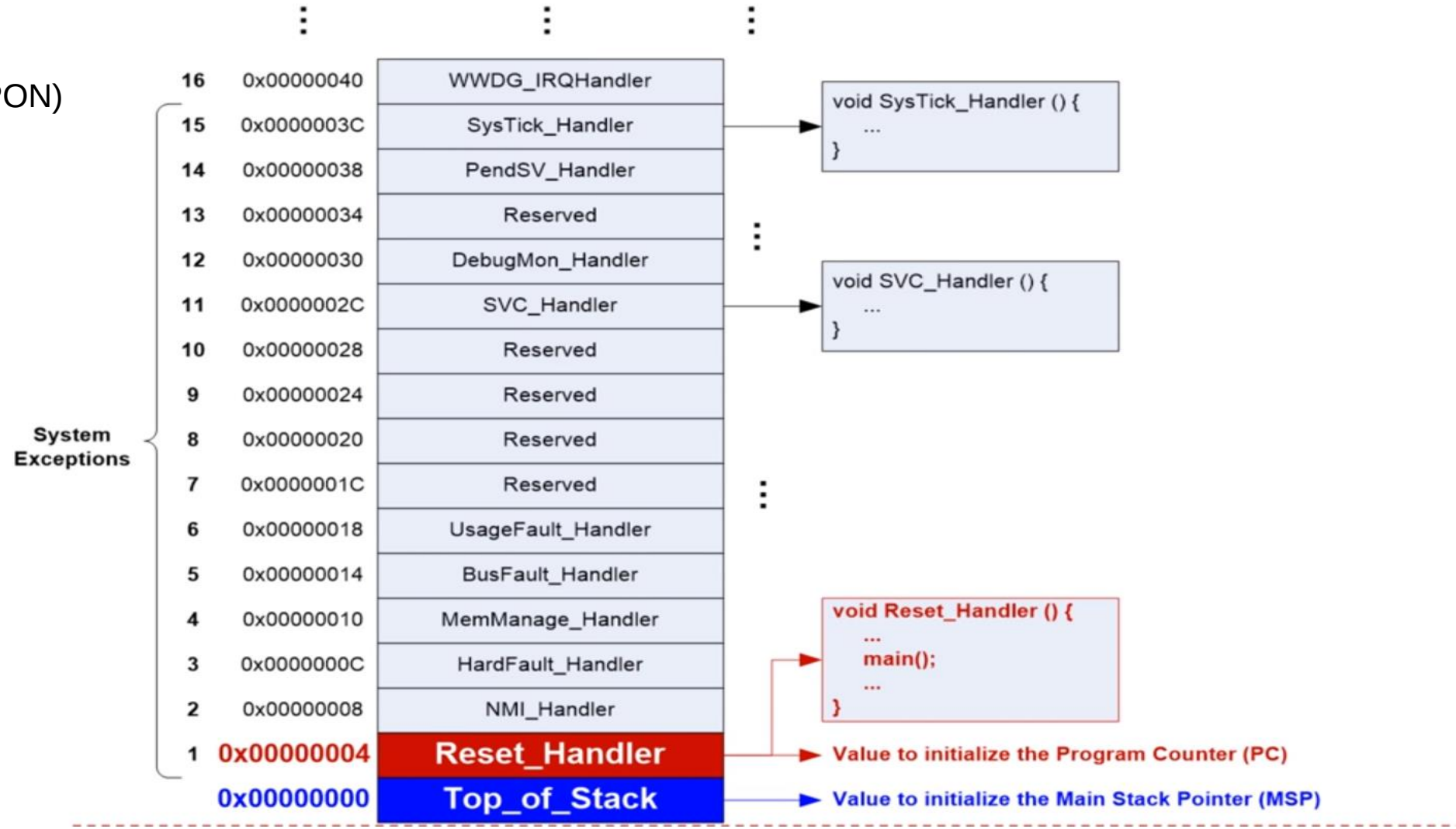
1. Vector table:

- The vector table contains the initialization value for the stack pointer, and the entry point addresses of each exception handler.
- The vector table is normally defined in the startup codes provided by the microcontroller vendors.
- By default, the vector table starts at memory address 0 and it will usually be either flash memory or ROM devices.
- However, in some applications can re-allocated vector table at run time by modify the Vector Table Offset Register (VTOR) in System control block(SCB).

Memory Address		Exception Number
0x0000004C	Interrupt#3 vector	19
0x00000048	Interrupt#2 vector	18
0x00000044	Interrupt#1 vector	17
0x00000040	Interrupt#0 vector	16
0x0000003C	SysTick vector	15
0x00000038	PendSV vector	14
0x00000034	Not used	13
0x00000030	Debug Monitor vector	12
0x0000002C	SVC vector	11
0x00000028	Not used	10
0x00000024	Not used	9
0x00000020	Not used	8
0x0000001C	Not used	7
0x00000018	Usage Fault vector	6
0x00000014	Bus Fault vector	5
0x00000010	MemManage vector	4
0x0000000C	HardFault vector	3
0x00000008	NMI vector	2
0x00000004	Reset vector	1
0x00000000	MSP initial value	0

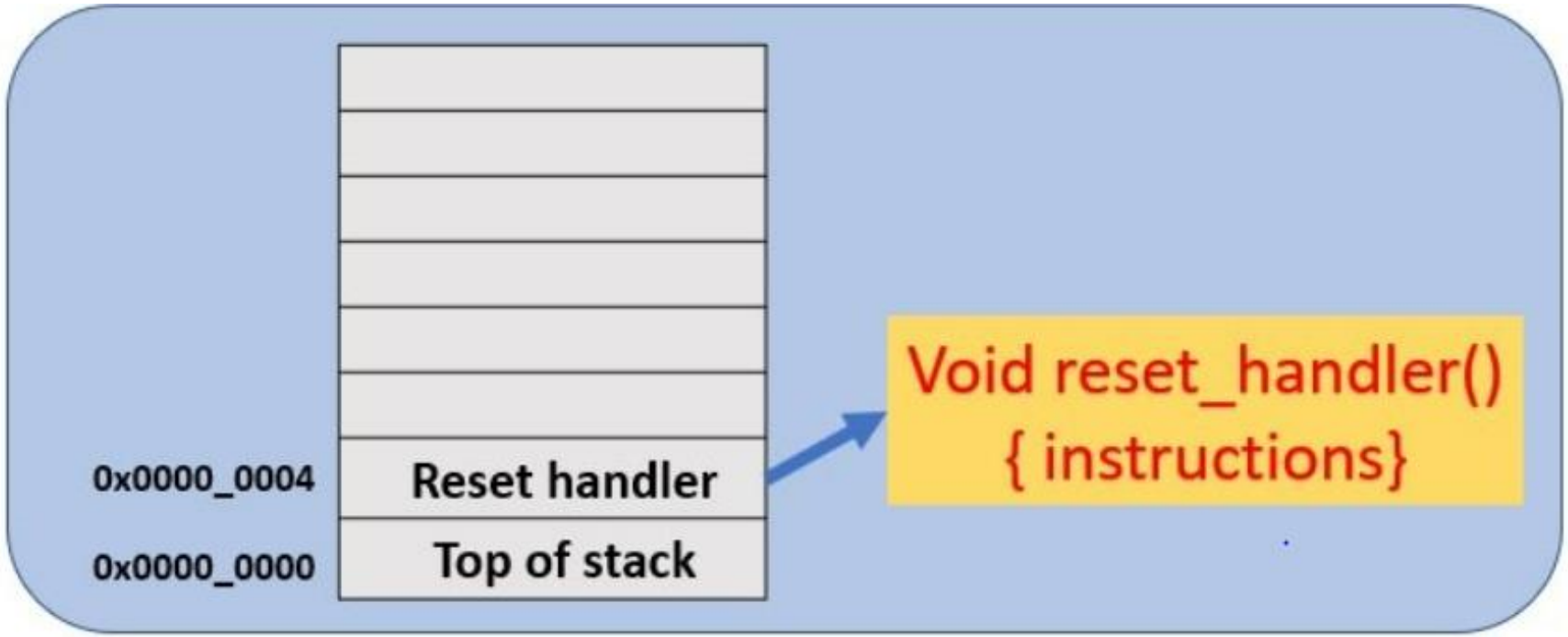
2. Booting process:

- Power supply(power on - PON)
- Hardware initialization
- Select boot region
- Init stack pointer
- Execute Reset_Handler()
- User initialization
- execute main()



2. Booting process:

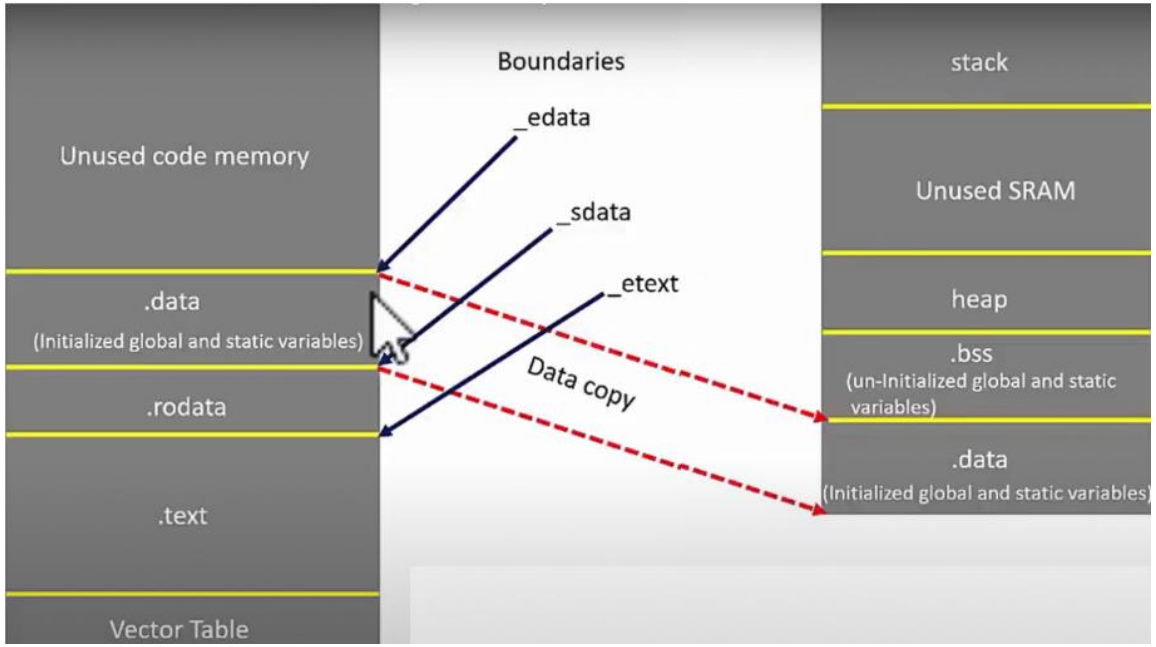
- PC is load with the value 0x00000000.
- The processor read the value at 0x00000000 and load this value into Main stack pointer(MSP).
- The processor read the value at 0x00000004 and load this value into PC (program counter), this value is actually address of reset handler.
- PC jump to reset handler function. Reset handler is just a C or assembly function written by user to carry out any initialization required.
- From reset handler, user call main function of the application.



© Copyright 2018 FPT Software

3. Reset_handler()

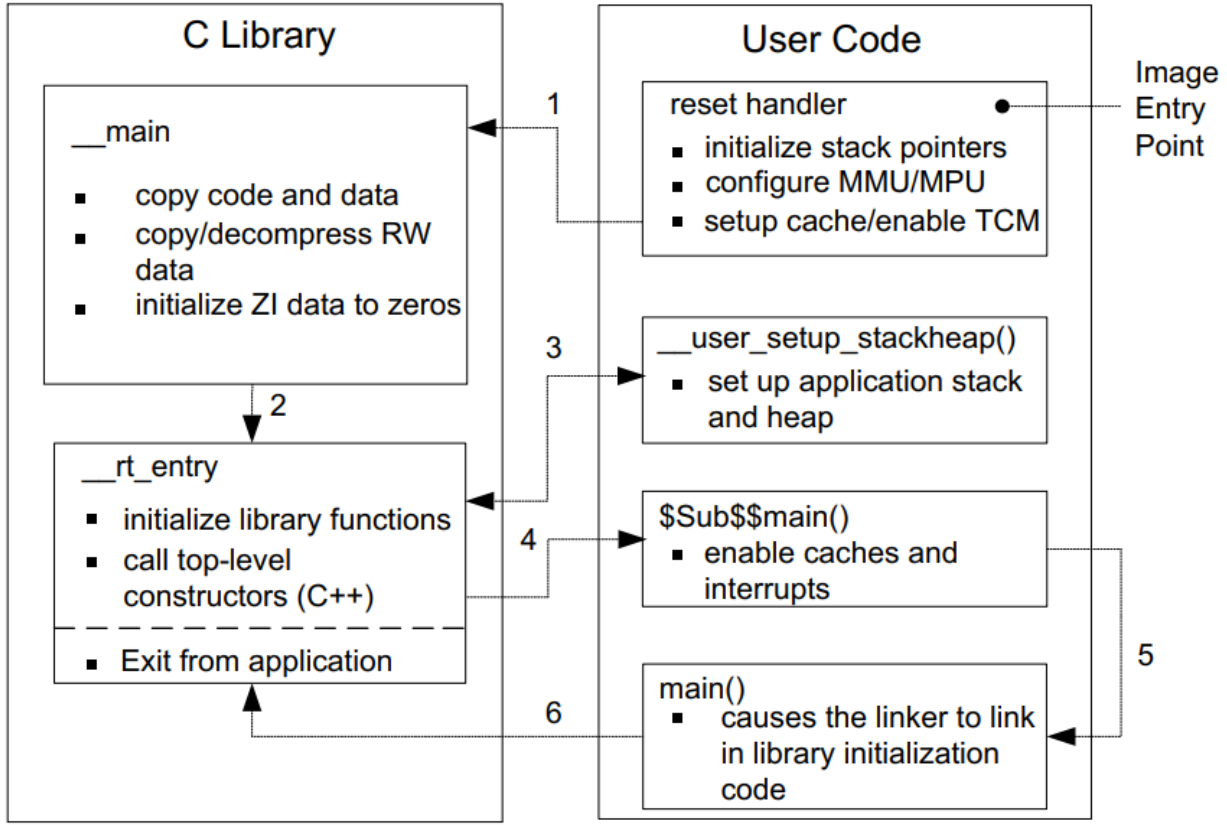
1. Disable all interrupts: All Interrupts Service will not be executed(E.g. Watchdog…)
2. Initialize data segment: Move data from data segment from ROM to RAM.
3. Initialize the .Bss segment: Reset value in bss section to zero.
4. Initialize stack segment: Reserve stack memory in RAM base on linker file.
5. Enable interrupts
6. Call main function



© Copyright 2018 FPT Software

4. **Initialization Sequence:**

- The following figure shows a possible initialization sequence for an embedded system based on an Arm architecture:

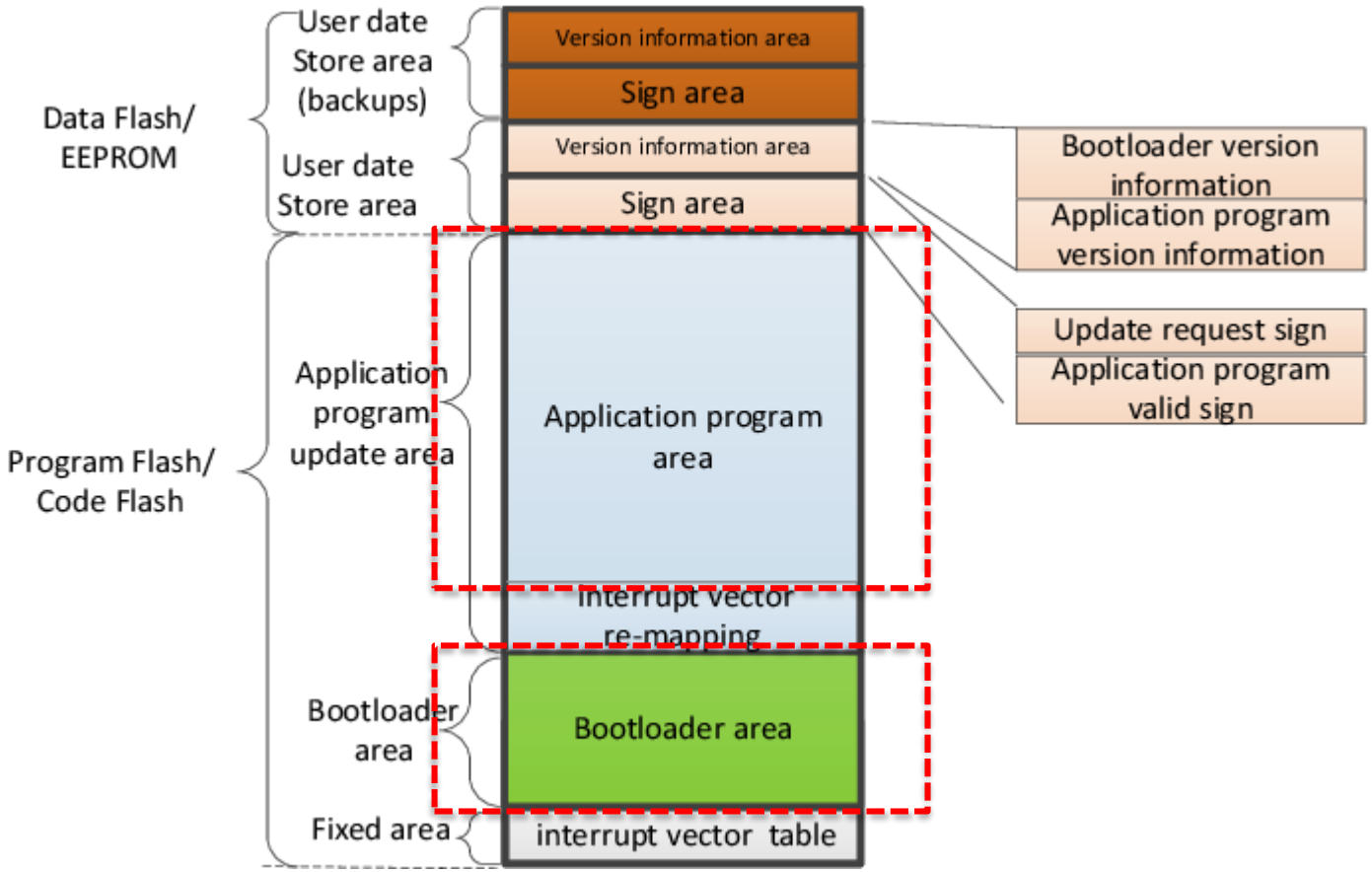


- Note: User can define the initialization sequence which not use the Arm library (__main() or __rt_entry()).

© Copyright 2018 FPT Software

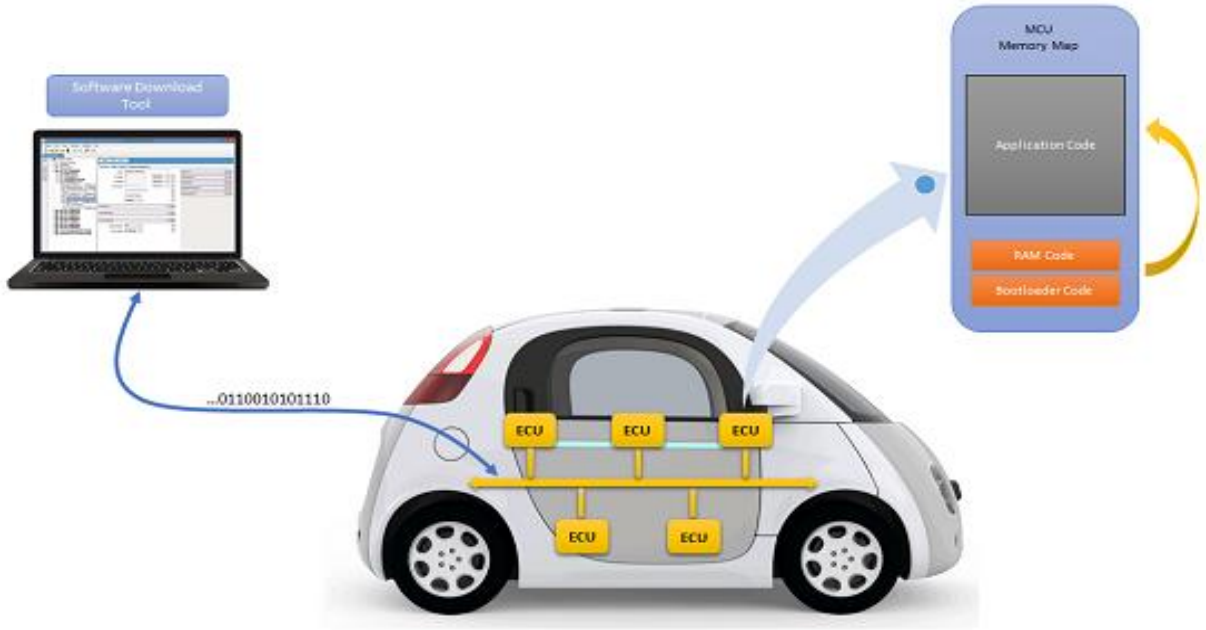
1. What is Bootloader?

- ✓ The bootloader is a program that boots the system or operating system that has been programmed and stored in the device's ROM/Flash.
- ✓ The bootloader is executed before the application.



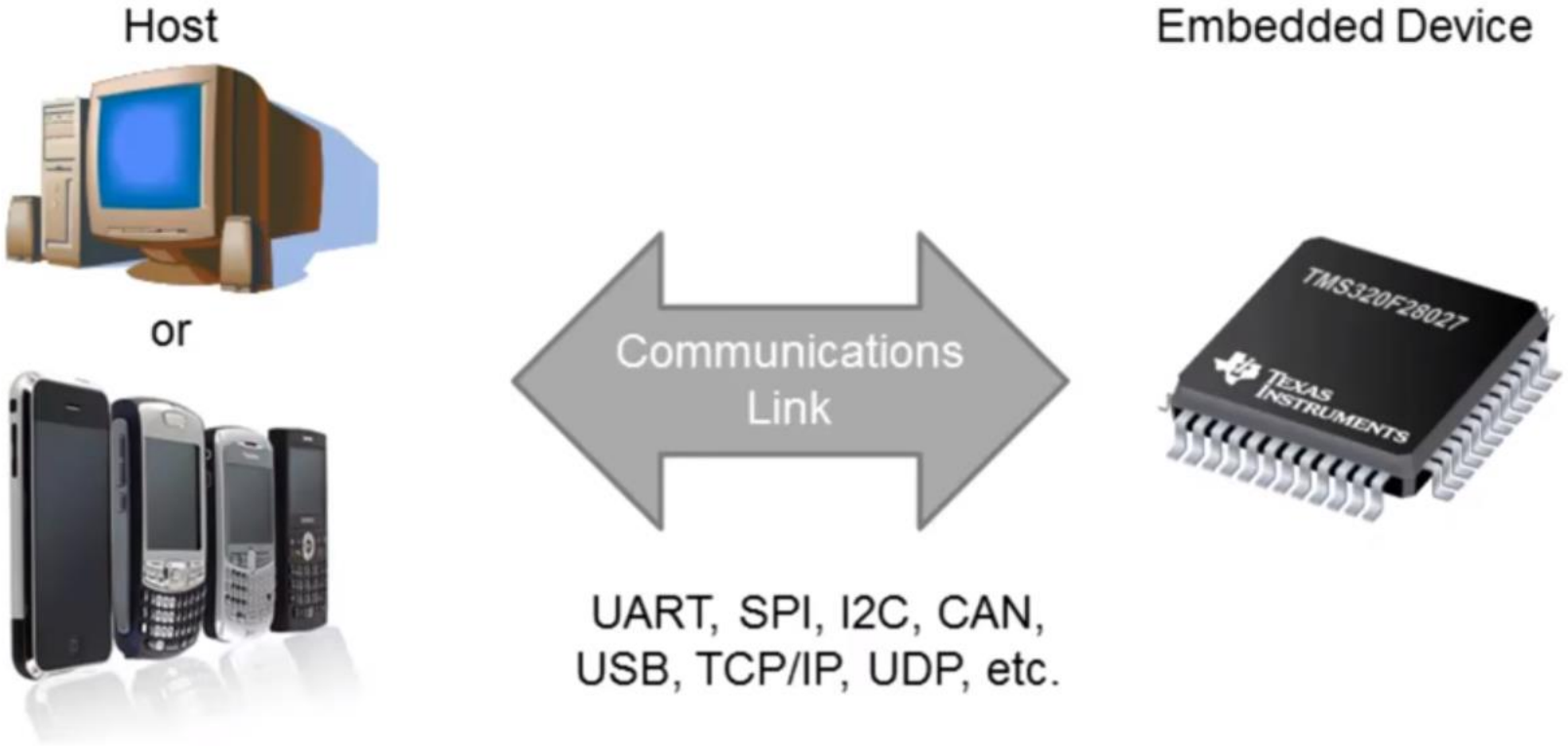
2. What is the purpose of Bootloader?

- Use to update the Application:
  - After the application has been corrected.
  - After the application is added the new feature.
  - After the customer changes the request (Requirement)…



3. The protocols that Bootloader uses:

- CAN
- LIN
- FlexRay
- Ethernet
- OTA
- ….



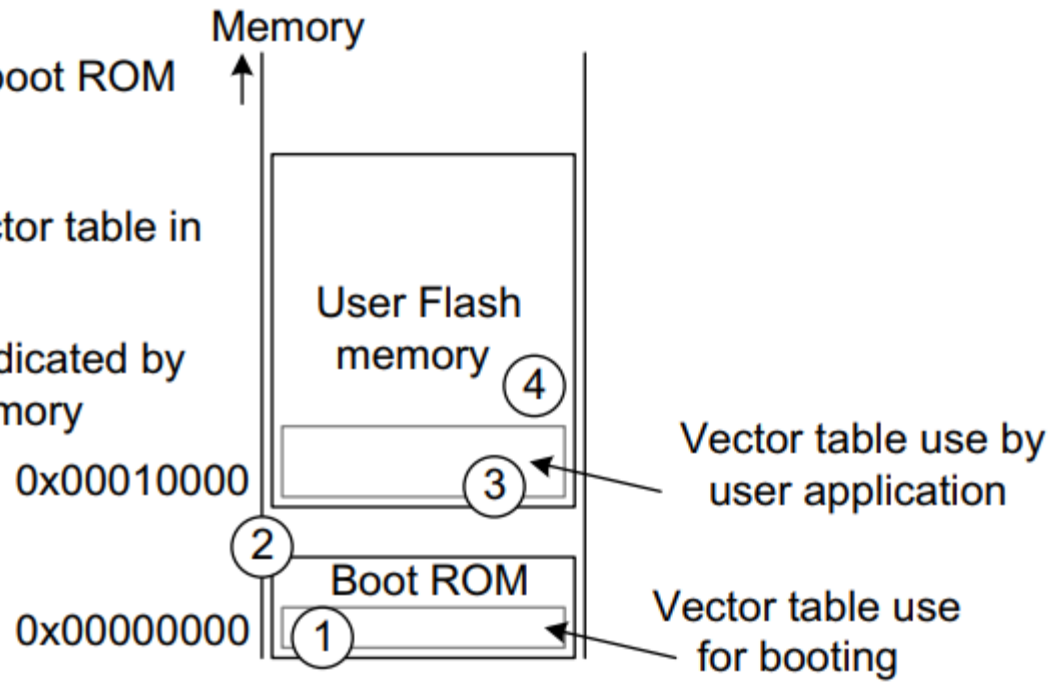
4. The switch mechanism from bootloader to application:

1) Boot up using vector table in boot ROM

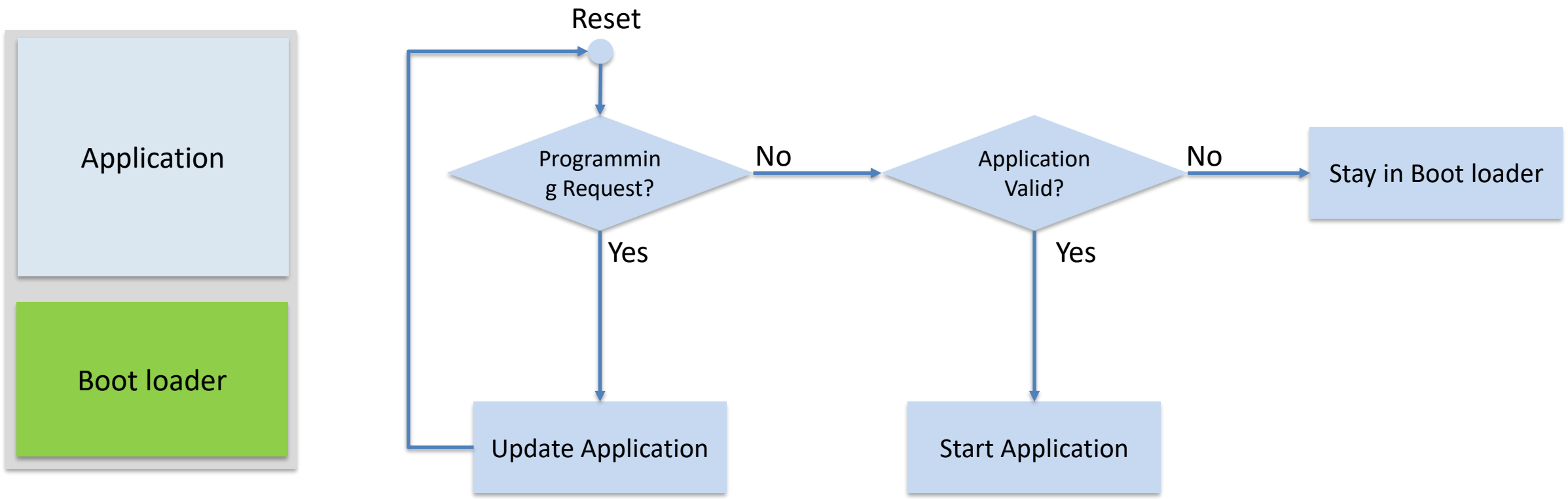
2) Carry out boot loader tasks

3) Program VTOR to point to vector table in user flash

4) Branch to the reset handler indicated by the vector table of user flash memory

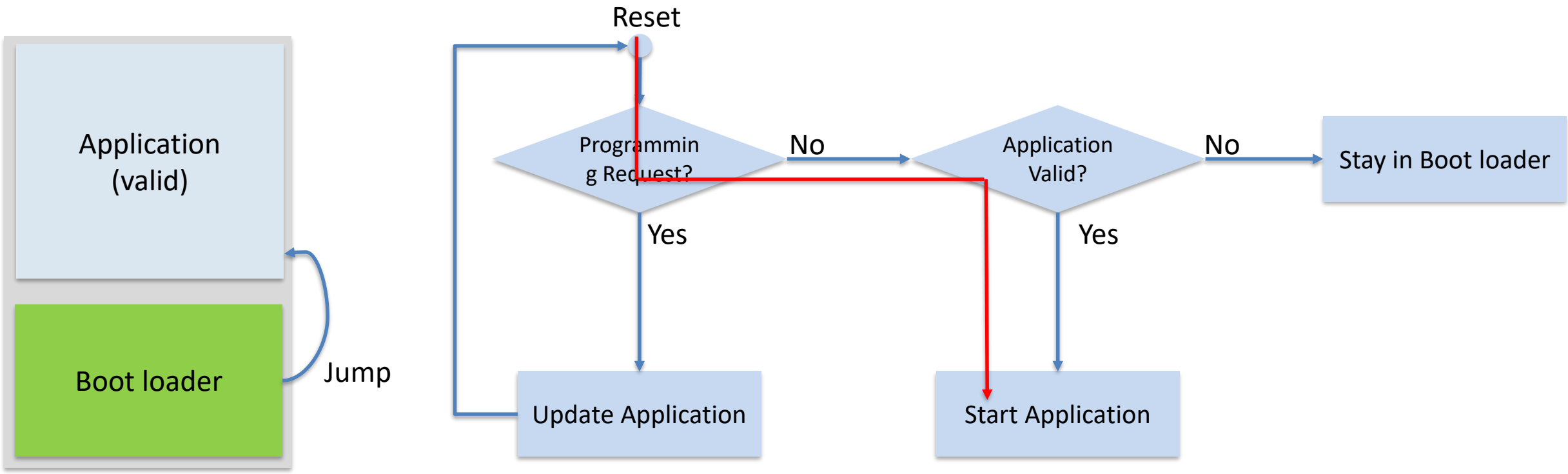


5. Boot Loader's activity in the simplest case : 1 Bootloader and 1 Application.

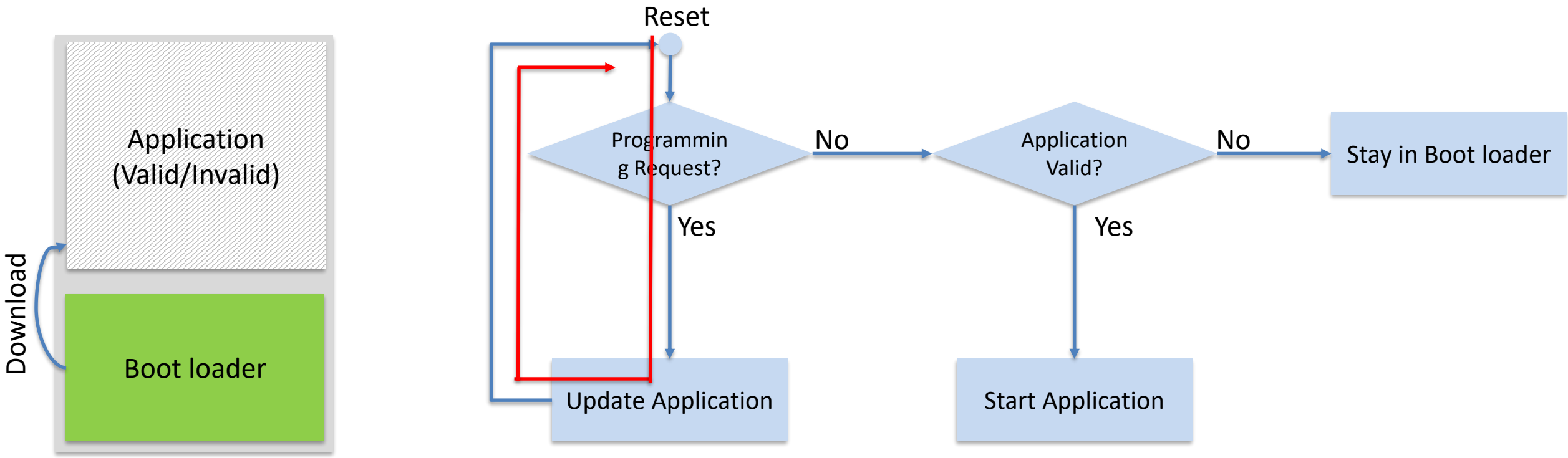


3. BOOT LOADER - MECHANISM

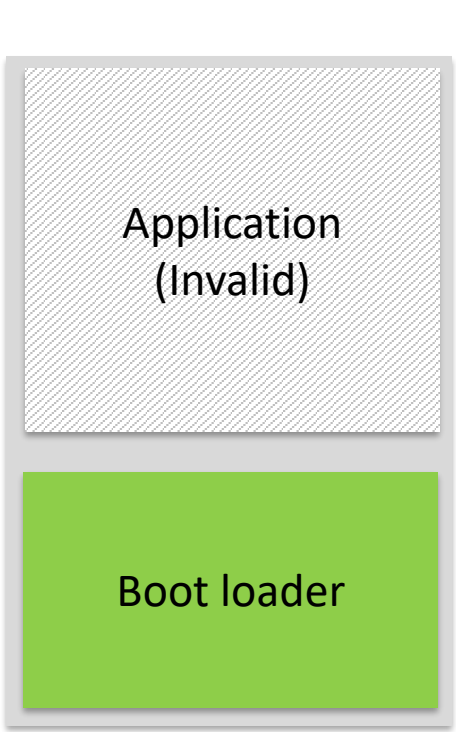
- Case 1: There is no update (Programming Request) and Application Valid.

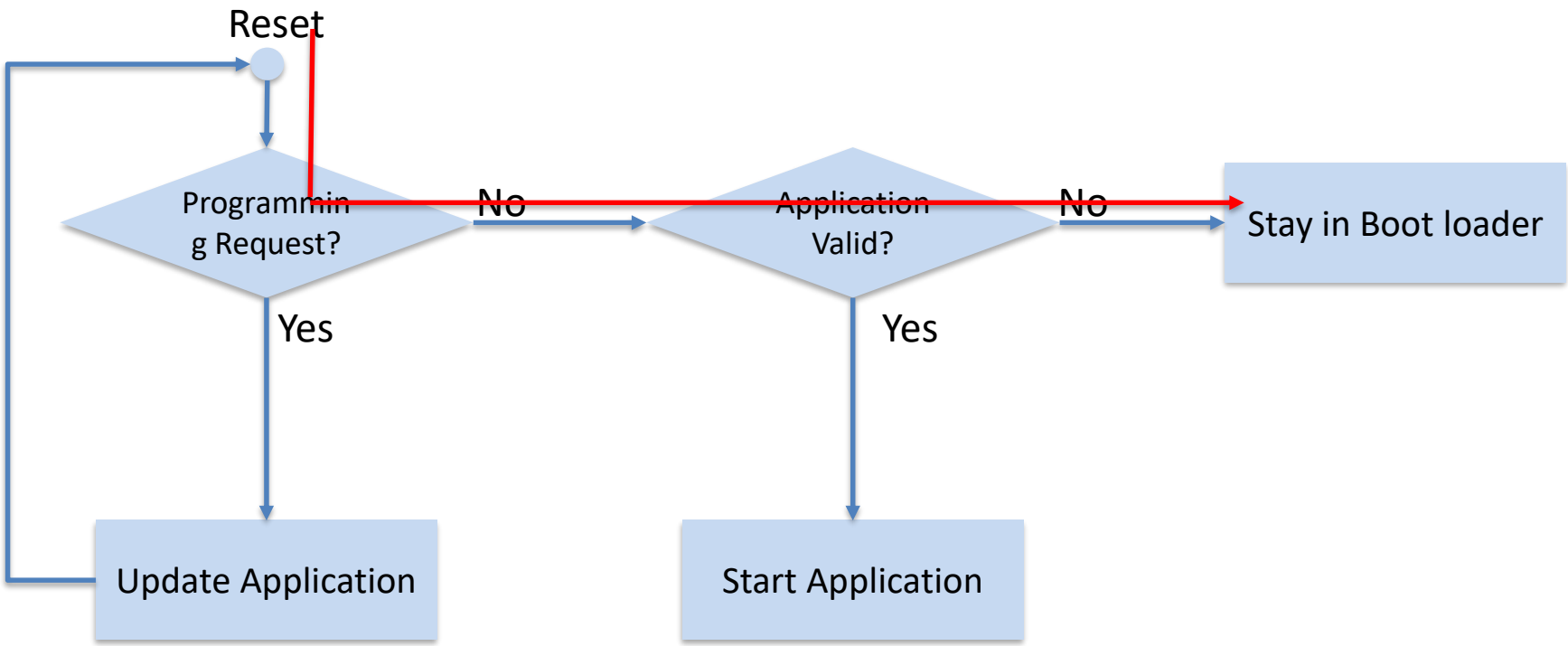


- Case 2: Programming request requires

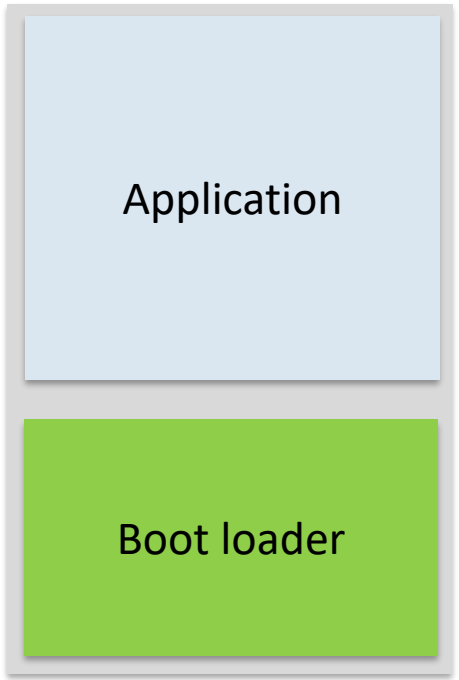


- Case 3: There is no update (Programming Request) and Application Invalid

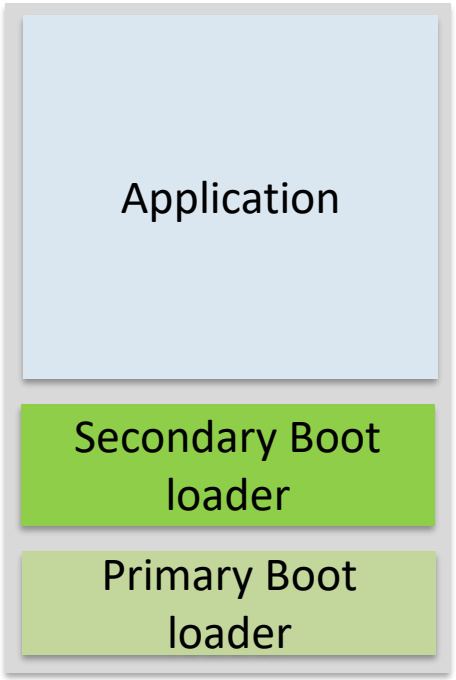




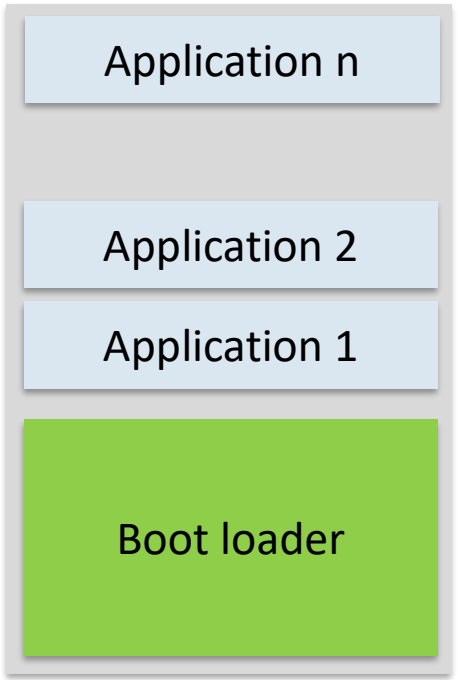
3. BOOT LOADER – EXAMPLE



1 boot loader
1 application



2 boot loader
1 application



1 boot loader
n applications

4. EXERCISE

- Question 1: What is PC? What is the function of PC?
- Question 2: Why does the user copy data or execution command from ROM to RAM in the boot process?
- Question 3: In the boot process, is the Initialize Stack needed? The purpose of the stack?
- Question 4: Indicate and evaluate the differences between bootloaders.
- Question 5: Try giving problems that might be encountered when making a bootloader SW for an electronic control unit.

© Copyright 2018 FPT Software

Q&A



© Copyright 2018 FPT Software